Lecture 29 — Strings II: Tries, Aho-Corasick & Suffix Arrays

> Year Curriculum · Prerequisites: Week 16 (BFS), Week 28 (hashing, prefix function, Z-function) · Estimated time: 2.5–3 hours

1. Overview

Last week every tool (hashing, KMP, Z) answered questions about **one pattern** against **one text**. This week we scale up in two directions:

1. **Many patterns.** A *trie* stores a dictionary so that all words sharing a prefix share storage and work. Adding KMP-style failure links to a trie gives the **Aho-Corasick automaton**, which finds every occurrence of every pattern in a single pass over the text.
2. **All substrings.** The **suffix array** sorts every suffix of a string. Because every substring is a prefix of some suffix, a sorted list of suffixes (plus the **LCP array**) lets us answer questions about all $O(n^2)$ substrings in near-linear time.

By the end you should be able to implement a trie (character and binary), Aho-Corasick with occurrence counting, an $O(n \log n)$ or $O(n \log^2 n)$ suffix array, and Kasai's LCP algorithm. Advanced suffix-structure applications (suffix automaton, LCP range-minimum tricks) come in Nationals lecture 6.

Notation: σ is the alphabet size (26 for lowercase), $|s|$ is length, $\Sigma|p_i|$ is the total length of all patterns.

2. Tries

2.1 Structure

A **trie** (prefix tree) is a rooted tree in which every edge carries one character, and the children of a node carry distinct characters. Each node corresponds to exactly one string: the concatenation of edge labels on the path from the root. We mark nodes where an inserted word ends.

```
words: car, cat, cart, dog
        (root)
        /    \
       c      d
```

```
      |      |
      a      o
     / \     |
    r*  t*   g*
    |
    t*
```

Inserting a word walks down from the root, creating missing children. The number of nodes is at most $1 + \Sigma|w_i|$ (each character creates at most one node), which is the bound you must size arrays by.

2.2 Implementation choices

- **Array of children** `int nxt[N][26]`: O(1) transitions, memory $N \cdot \sigma \cdot 4$ bytes. For $N = 10^6$ that is 104 MB, often too much; for $N = 2 \cdot 10^5$ it is ~21 MB, fine.
- **`map<char,int>` per node**: memory proportional to actual edges, O(log σ) per step. Use for large alphabets.
- **Vector of `array<int,26>`** grown with `push_back`: no fixed maximum, same speed as the static array.

Useful per-node counters:

- `endCnt[v]` — how many inserted words end at v (handles duplicates).
- `passCnt[v]` — how many inserted words pass through v; then "how many words have prefix p" is `passCnt` of the node reached by p.

Each insertion or lookup is O(|s|).

2.3 Trie + DP: Word Combinations

CSES 1731: given s ($|s| \le 5000$) and $k \le 10^5$ dictionary words (total length $\le 10^6$), count the ways to write s as a concatenation of dictionary words, modulo 10^9+7.

Let `dp[i]` be the number of ways to build the suffix `s[i..n)`, with `dp[n] = 1`. For each i (from right to left) walk the trie along `s[i], s[i+1], …`; whenever the node after reading `s[i..j)` is the end of a word, add `dp[j]`. Stop as soon as the trie has no child. Each start walks at most $\min(n - i, \text{maxWordLength})$ steps, so the total is $O(n \cdot L) \le 2.5 \cdot 10^7$ in the worst case — fast. Compare with testing each word at each position: $O(n \cdot \Sigma|w_i|)$, hopeless.

2.4 Binary tries for XOR

Write every number as a fixed-width bit string, **most significant bit first**, and insert it into a trie with alphabet {0, 1}. To maximize `x XOR y` over stored y, walk from the root and at bit b prefer the child whose bit is the opposite of bit b of x. This greedy is optimal because $2^b > 2^{b-1} + \ldots + 1$: winning a higher bit beats any combination of lower bits.

To support deletion, store `cnt[v]` = number of stored values passing through v; insert adds 1 along the path, delete subtracts 1, and the query treats a child with `cnt = 0` as absent. Each operation is O(B) with B = 30 or 31 bits. This solves CF 706D (Vasiliy's Multiset) directly and is also the key tool for "maximum XOR subarray" (insert prefix XORs as you sweep).

3. Aho-Corasick

3.1 Failure links

Recall the KMP prefix function: when the next text character does not extend the current match, we fall back to the longest proper border of the matched part, which is still a valid partial match. Aho-Corasick does exactly this on a trie of all patterns.

For a trie node v representing string str(v), define the **failure (suffix) link** `link[v]` as the node representing the **longest proper suffix of str(v) that is also a node of the trie** (i.e., a prefix of some pattern). The root's link is the root, and every depth-1 node links to the root.

Example with patterns `he, she, his, hers`:

node	longest proper suffix in trie	link
she	"he"	he
sh	"h"	h
his	"s"	s
hers	"s" ("ers", "rs" not in trie)	s
he, hi, her	none	root

When the scan of the text reaches node `she`, the patterns ending at the current text position are exactly the terminal nodes on the chain `she → he → root`: both "she" and "he" match.

3.2 Computing links: BFS

If u = child(v, c), then link[u] is found by following v's link chain until some node w has a child with character c: link[u] = child(w, c), or root if none exists. Because link[v] is a strictly shorter string than v, links must be known for all shorter strings first, so we process nodes in **BFS order** (by depth). DFS order is a classic bug.

3.3 The goto automaton

Following failure chains during the scan is amortized O(|t|) for one text but awkward. The cleaner technique is to fill in **all** transitions:

```
go(v, c) = child(v, c)          if that child exists
         = go(link[v], c)       otherwise          (go(root, c) = root if missing)
```

In BFS order, `go(link[v], c)` is already computed when we process v, so every transition is O(1). Moreover, the link of a real child becomes simply `link[child(v,c)] = go(link[v], c)`. Total build cost: $O(\Sigma|p_i| \cdot \sigma)$ time and memory. The result is a deterministic finite automaton: scanning the text is `v = go[v][t[i]]`, O(1) per character.

3.4 Counting occurrences of every pattern

At each text position the current state v tells us that every terminal node on v's link chain matched. Walking the chain at every position can cost $O(|t| \cdot \sqrt{\Sigma|p_i|})$ (e.g., patterns a, aa, aaa, …). Instead:

1. During the scan, increment `hit[v]` for the state after each character.
2. The links form a tree rooted at the root (the **fail tree**). The number of times node u's string occurred in the text is the sum of `hit` over u's fail-subtree. Compute it by processing nodes in **reverse BFS order** and doing `hit[link[v]] += hit[v]` (a child in the fail tree is always deeper in the trie, hence later in BFS order).
3. The answer for pattern i is `hit[end[i]]`, where `end[i]` is the node at which pattern i was inserted. Store this per pattern, so duplicate patterns are handled automatically.

Total time $O(|t| + \Sigma|p_i| \cdot \sigma)$.

3.5 Other uses

- **Does the text contain any pattern?** Propagate `bad[v] |= bad[link[v]]` in BFS order, then check whether the scan visits a bad state.
- **List every occurrence:** keep a *dictionary link* to the nearest terminal node on the link chain; following those visits only actual matches, giving O(|t| + #matches).
- **Count strings of length L avoiding all patterns:** DP over (length, automaton state), skipping bad states: O(L · #nodes · σ). For huge L use matrix exponentiation (Nationals 7).

4. Suffix arrays

4.1 Definition

The **suffix array** of s (length n) is the permutation SA of 0..n−1 such that the suffixes s[SA[0]..], s[SA[1]..], … are in increasing lexicographic order. For `banana`:

rank	SA	suffix
0	5	a

rank	SA	suffix
1	3	ana
2	1	anana
3	0	banana
4	4	na
5	2	nana

Every substring of s is a prefix of some suffix, and all suffixes starting with a given pattern p form a **contiguous block** of the SA, which we can find by binary search in O(|p| log n).

4.2 Prefix doubling

Sorting n suffixes with string comparisons costs O(n log n) comparisons of O(n) each: O(n^2 log n). Prefix doubling sorts by the first 1, 2, 4, … characters:

- Let `rk_k[i]` be the rank of `s[i..i+k)` (truncated at the end of the string) among all such strings; equal strings get equal ranks.
- The length-2k prefix of suffix i is the length-k prefix of i followed by the length-k prefix of i + k. So suffixes are ordered by the pair `(rk_k[i], rk_k[i+k])`, with −1 when i + k ≥ n (a shorter string is smaller).
- After sorting by pairs, assign new ranks by scanning and incrementing whenever the pair changes.
- Stop when all ranks are distinct. At most $\lceil \log_2 n \rceil + 1$ rounds.

With `std::sort` each round is O(n log n), total **O(n $\log^2$ n)** — fine for n ≤ $2 \cdot 10^5$ to $5 \cdot 10^5$. Replacing the comparison sort by two counting-sort passes (radix sort on the pair) gives **O(n log n)**; the reference code in Section 5 includes that version, written in the common "cyclic shifts with sentinel" style.

4.3 The LCP array and Kasai's algorithm

Define `lcp[i]` = length of the longest common prefix of suffixes SA[i] and SA[i+1] (for i = 0..n−2). For `banana`: lcp = [1, 3, 0, 0, 2].

Kasai's algorithm computes it in O(n). Let rank[i] be the SA position of suffix i. Process suffixes in **text order** i = 0, 1, 2, …, maintaining h. Claim: if the suffix just before suffix i in the SA shares h characters with it and h > 0, then the suffix just before suffix i + 1 shares at least h − 1 characters with it. (Drop the first character from both suffixes: they stay in the same order and share h − 1 characters, and anything between them in the SA shares at least that much.) So we start the character comparison from h − 1 instead of 0. h increases at most 2n times overall, giving O(n).

4.4 First applications

- **Number of distinct substrings.** Suffix SA[i] has n − SA[i] prefixes; the first lcp[i−1] of them already appeared as prefixes of SA[i−1]. So the count is n(n+1)/2 − Σ lcp. For banana: 21 − 6 = 15.
- **Longest repeated substring:** max(lcp), achieved at the adjacent pair.
- **Longest common substring of A and B:** build the SA of `A + '#' + B` (with '#' not in either string) and take the max lcp over adjacent pairs where one suffix starts in A and the other in B.
- **Pattern occurrences:** two binary searches for the block of suffixes having p as a prefix.

5. Reference implementations

5.1 Trie + DP — CSES 1731 Word Combinations

```cpp
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1'000'000'007;
vector<array<int, 26>> nxt;
vector<char> isEnd;

int newNode() {
    nxt.push_back({});
    nxt.back().fill(0);
    isEnd.push_back(0);
    return (int)nxt.size() - 1;
}

void insertWord(const string& w) {
    int v = 0;
    for (char ch : w) {
        int c = ch - 'a';
        if (!nxt[v][c]) { int u = newNode(); nxt[v][c] = u; }
        v = nxt[v][c];
    }
    isEnd[v] = 1;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    int k;
    cin >> s >> k;
    newNode();  // root = 0
    for (int i = 0; i < k; i++) {
        string w;
        cin >> w;
```

```
        insertWord(w);
    }
    int n = s.size();
    vector<long long> dp(n + 1, 0);
    dp[n] = 1;
    for (int i = n - 1; i >= 0; i--) {
        int v = 0;
        for (int j = i; j < n; j++) {
            v = nxt[v][s[j] - 'a'];
            if (!v) break;
            if (isEnd[v]) dp[i] = (dp[i] + dp[j + 1]) % MOD;
        }
    }
    cout << dp[0] << "\n";
}
```

Note the `int u = newNode();` before assigning: writing `nxt[v][c] = newNode();` is unsafe pre-C++17 evaluation order semantics with a reallocating vector. Sample: `ababc` with words `ab abab c cb` gives 2.

5.2 Binary trie — CF 706D Vasiliy's Multiset

```
#include <bits/stdc++.h>
using namespace std;

const int B = 30;
vector<array<int, 2>> ch;
vector<int> cnt;

void update(int x, int delta) {
    int v = 0;
    for (int b = B - 1; b >= 0; b--) {
        int bit = (x >> b) & 1;
        if (!ch[v][bit]) {
            ch[v][bit] = ch.size();
            ch.push_back({0, 0});
            cnt.push_back(0);
        }
        v = ch[v][bit];
        cnt[v] += delta;
    }
}

int maxXor(int x) {  // assumes the multiset is non-empty
    int v = 0, res = 0;
    for (int b = B - 1; b >= 0; b--) {
        int want = ((x >> b) & 1) ^ 1;
        int u = ch[v][want];
        if (u && cnt[u] > 0) { res |= 1 << b; v = u; }
        else v = ch[v][want ^ 1];
    }
    return res;
```

```
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    ch.push_back({0, 0});
    cnt.push_back(0);
    update(0, +1);  // the multiset initially contains 0
    int q;
    cin >> q;
    while (q--) {
        char op;
        int x;
        cin >> op >> x;
        if (op == '+') update(x, +1);
        else if (op == '-') update(x, -1);
        else cout << maxXor(x) << "\n";
    }
}
```

5.3 Aho-Corasick — CSES 2103 Counting Patterns

```
#include <bits/stdc++.h>
using namespace std;

struct AhoCorasick {
    static const int A = 26;
    vector<array<int, A>> nxt;
    vector<int> link, order;

    AhoCorasick() { addNode(); }
    int addNode() {
        nxt.push_back({});
        nxt.back().fill(-1);
        link.push_back(0);
        return (int)nxt.size() - 1;
    }
    int insert(const string& s) {  // returns the terminal node
        int v = 0;
        for (char ch : s) {
            int c = ch - 'a';
            if (nxt[v][c] == -1) { int u = addNode(); nxt[v][c] = u; }
            v = nxt[v][c];
        }
        return v;
    }
    void build() {
        queue<int> q;
        for (int c = 0; c < A; c++) {
            int u = nxt[0][c];
            if (u == -1) nxt[0][c] = 0;
            else { link[u] = 0; q.push(u); }
        }
```

```
        order.push_back(0);
        while (!q.empty()) {
            int v = q.front(); q.pop();
            order.push_back(v);
            for (int c = 0; c < A; c++) {
                int u = nxt[v][c];
                if (u == -1) nxt[v][c] = nxt[link[v]][c];
                else { link[u] = nxt[link[v]][c]; q.push(u); }
            }
        }
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string t;
    int k;
    cin >> t >> k;
    AhoCorasick ac;
    vector<int> endNode(k);
    for (int i = 0; i < k; i++) {
        string p;
        cin >> p;
        endNode[i] = ac.insert(p);
    }
    ac.build();
    vector<long long> hit(ac.nxt.size(), 0);
    int v = 0;
    for (char ch : t) {
        v = ac.nxt[v][ch - 'a'];
        hit[v]++;
    }
    for (int i = (int)ac.order.size() - 1; i > 0; i--) {
        int u = ac.order[i];
        hit[ac.link[u]] += hit[u];
    }
    for (int i = 0; i < k; i++) cout << hit[endNode[i]] << "\n";
}
```

Sample: text `aybabtu`, patterns `bab abc a` → output `1 0 2`.

5.4 Suffix array (both versions) + Kasai — CSES 2105 Distinct Substrings

```
#include <bits/stdc++.h>
using namespace std;

// O(n log^2 n): simple and usually fast enough.
vector<int> suffixArraySimple(const string& s) {
    int n = s.size();
    vector<int> sa(n), rk(n), tmp(n);
```

```cpp
    for (int i = 0; i < n; i++) { sa[i] = i; rk[i] = (unsigned char)s[i]; }
    for (int k = 1;; k <<= 1) {
        auto key = [&](int i) { return i + k < n ? rk[i + k] : -1; };
        auto cmp = [&](int a, int b) {
            return rk[a] != rk[b] ? rk[a] < rk[b] : key(a) < key(b);
        };
        sort(sa.begin(), sa.end(), cmp);
        tmp[sa[0]] = 0;
        for (int i = 1; i < n; i++) tmp[sa[i]] = tmp[sa[i - 1]] + cmp(sa[i - 1], sa[i]);
        rk = tmp;
        if (rk[sa[n - 1]] == n - 1) break;
    }
    return sa;
}

// O(n log n): sort cyclic shifts of s + '\0' with counting sort.
vector<int> suffixArrayFast(string s) {
    s.push_back('\0');                // sentinel smaller than every character
    int n = s.size(), classes = 0;
    const int ALPHA = 256;
    vector<int> p(n), c(n), cnt(max(ALPHA, n), 0), pn(n), cn(n);
    for (char ch : s) cnt[(unsigned char)ch]++;
    for (int i = 1; i < ALPHA; i++) cnt[i] += cnt[i - 1];
    for (int i = n - 1; i >= 0; i--) p[--cnt[(unsigned char)s[i]]] = i;
    c[p[0]] = 0; classes = 1;
    for (int i = 1; i < n; i++) {
        if (s[p[i]] != s[p[i - 1]]) classes++;
        c[p[i]] = classes - 1;
    }
    for (int h = 1; h < n; h <<= 1) {
        for (int i = 0; i < n; i++) pn[i] = (p[i] - h + n) % n;  // sorted by 2nd half
        fill(cnt.begin(), cnt.begin() + classes, 0);
        for (int i = 0; i < n; i++) cnt[c[pn[i]]]++;
        for (int i = 1; i < classes; i++) cnt[i] += cnt[i - 1];
        for (int i = n - 1; i >= 0; i--) p[--cnt[c[pn[i]]]] = pn[i];  // stable by 1st
half
        cn[p[0]] = 0; classes = 1;
        for (int i = 1; i < n; i++) {
            pair<int, int> cur = {c[p[i]], c[(p[i] + h) % n]};
            pair<int, int> prv = {c[p[i - 1]], c[(p[i - 1] + h) % n]};
            if (cur != prv) classes++;
            cn[p[i]] = classes - 1;
        }
        c.swap(cn);
        if (classes == n) break;
    }
    p.erase(p.begin());               // drop the sentinel suffix
    return p;
}

vector<int> lcpArray(const string& s, const vector<int>& sa) {
    int n = s.size(), h = 0;
    vector<int> rk(n), lcp(max(n - 1, 0));
    for (int i = 0; i < n; i++) rk[sa[i]] = i;
    for (int i = 0; i < n; i++) {
```

```
        if (rk[i] == 0) { h = 0; continue; }
        int j = sa[rk[i] - 1];
        while (i + h < n && j + h < n && s[i + h] == s[j + h]) h++;
        lcp[rk[i] - 1] = h;
        if (h > 0) h--;
    }
    return lcp;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    vector<int> sa = suffixArrayFast(s);
    assert(sa == suffixArraySimple(s));
    vector<int> lcp = lcpArray(s, sa);
    long long n = s.size();
    long long ans = n * (n + 1) / 2;
    for (int x : lcp) ans -= x;
    cout << ans << "\n";
}
```

For input `banana` both builders give SA = [5, 3, 1, 0, 4, 2], lcp = [1, 3, 0, 0, 2], and the program prints 15. (In a contest keep just one builder; the `assert` is here to show they agree.)

6. Common mistakes

1. **Undersized trie.** The node count is bounded by total length + 1, not by the number of words. Size accordingly, or grow a vector.
2. **Memory blow-up.** `int nxt[1e6][26]` is 104 MB. Use a `map` for sparse/large alphabets, or reduce σ.
3. **Reallocation bugs.** `nxt[v][c] = newNode();` where `newNode` pushes onto the same vector: before C++17 the left side could be evaluated first and then invalidated. Assign the new index to a local variable first; it is clearer anyway.
4. **Wrong link order.** Suffix links must be computed in BFS order (by depth). A DFS fills links using links that are not yet computed.
5. **Walking link chains per character.** Gives O(|t| · depth) in the worst case (pattern set `a, aa, aaa, …`). Aggregate over the fail tree instead.
6. **Duplicate patterns.** If you store "pattern id at node", the second copy overwrites the first. Store the terminal node for each pattern.
7. **Suffix-array off-by-one.** The second key `rk[i + k]` does not exist when i + k ≥ n; it must compare as smaller than any real rank. In the cyclic version, forgetting the sentinel makes the cyclic order differ from the suffix order (e.g., for `aa`).
8. **LCP indexing.** Decide once whether `lcp[i]` pairs SA[i] with SA[i+1] or with SA[i−1], and stick to it.

9. **Overflow.** The number of distinct substrings of a string with n = 10^5 is ~$5 \cdot 10^9$: use `long long`.

7. Practice problems

Tries

- CSES 1731 — Word Combinations (https://cses.fi/problemset/task/1731)
- Codeforces 706D — Vasiliy's Multiset (https://codeforces.com/problemset/problem/706/D)
- Codeforces 923C — Perfect Security (https://codeforces.com/problemset/problem/923/C)
- Codeforces 455B — A Lot of Games (https://codeforces.com/problemset/problem/455/B) — trie + game DP, a preview of Week 33

Aho-Corasick

- CSES 2102 — Finding Patterns (https://cses.fi/problemset/task/2102)
- CSES 2103 — Counting Patterns (https://cses.fi/problemset/task/2103)
- CSES 2104 — Pattern Positions (https://cses.fi/problemset/task/2104)
- Kattis — stringmultimatching (https://open.kattis.com/problems/stringmultimatching)
- Codeforces 1202E — You Are Given Some Strings... (https://codeforces.com/problemset/problem/1202/E)

Suffix arrays

- Kattis — suffixsorting (https://open.kattis.com/problems/suffixsorting)
- CSES 2105 — Distinct Substrings (https://cses.fi/problemset/task/2105)
- CSES 2106 — Repeating Substring (https://cses.fi/problemset/task/2106)
- SPOJ SUBST1 — New Distinct Substrings (https://www.spoj.com/problems/SUBST1/)

8. Further reading

- cp-algorithms: "Trie" / "Aho-Corasick algorithm" (https://cp-algorithms.com/string/aho_corasick.html) and "Suffix Array" (https://cp-algorithms.com/string/suffix-array.html)
- Codeforces EDU course "Suffix Array" (https://codeforces.com/edu/course/2) — step-by-step lessons with practice problems
- A. Aho, M. Corasick, "Efficient string matching: an aid to bibliographic search", CACM 1975
- T. Kasai et al., "Linear-time longest-common-prefix computation in suffix arrays and its applications", CPM 2001
- Antti Laaksonen, *Competitive Programmer's Handbook*, chapter 26 (string algorithms)